



Sprint 11

Vactory - Decoupled

Réalisé par :

Sara Akharraz

Encadrants :

M.Abdelmajid Bendrif

M.Hamza Bahlaouane

Entreprise :

Void

Objectif :

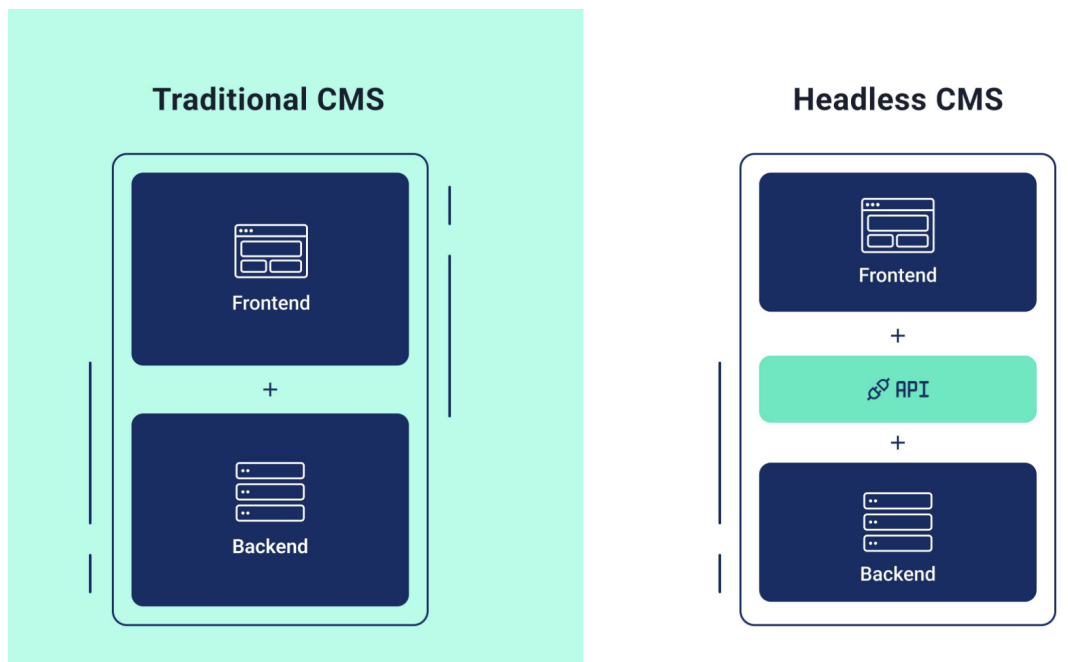
- Découvrir et maîtriser l'installation et le fonctionnement du Vactory Starter Kit en mode découplé comme backend pour Next.js, tout en comprenant ses principes de base et ses enjeux internes.

1. Problèmes liés à l'architecture découplée de Drupal :

1.1. Monolith vs Decoupled CMS :

Un CMS monolithique : regroupe dans un seul système la gestion du contenu et son affichage.

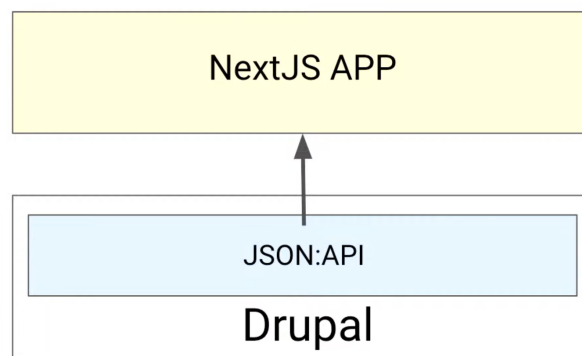
Un CMS découplé : sépare la partie back-end (création et stockage du contenu) de la partie front-end, permettant de diffuser ce contenu via des APIs vers différentes interfaces.



1.2. Les Types d'un Decoupled CMS :

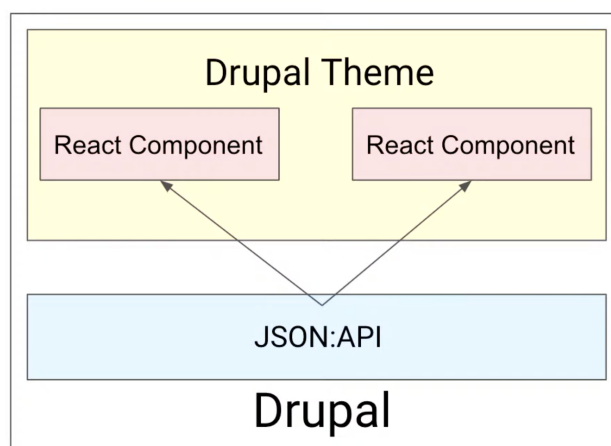
Fully decoupled : le back-end et le front-end sont totalement séparés et communiquent uniquement via des APIs, sans aucune dépendance entre les deux.

Fully Decoupled



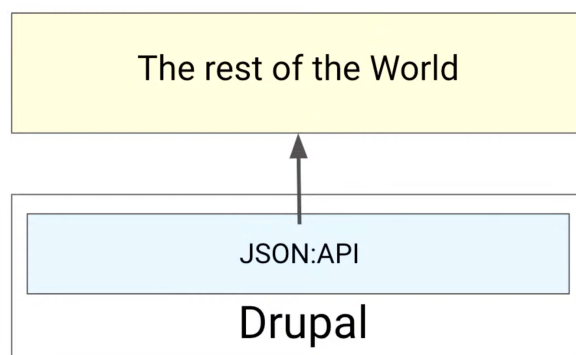
Progressive decoupled : le CMS reste principalement monolithique mais permet d'intégrer des composants front-end modernes (comme React ou Vue) en parallèle pour enrichir l'interface.

Progressive Decoupled



Headless : le CMS est utilisé uniquement comme back-end de gestion de contenu, sans couche de présentation, et le contenu est consommé via API par n'importe quel front-end.

Headless



Source

1.3. Problèmes du Drupal découplé :

1.3.1 REST vs JSON :API vs GraphQL

- **Efficacité des requêtes** : capacité de l'API à fournir uniquement les données nécessaires avec un minimum de requêtes.
- **Documentation de l'API** : facilité avec laquelle les développeurs peuvent comprendre, explorer et utiliser l'API.
- **Simplicité opérationnelle** : facilité de mise en place, d'utilisation et de maintenance de l'API au quotidien.
- **Écriture des données** : manière dont l'API permet de créer, modifier et supprimer des données (CRUD).

Résumé :

	REST	JSON:API	GraphQL
<u>Request efficiency</u>	Poor; multiple requests are needed to satisfy common needs. Responses are bloated.	Excellent; a single request is usually sufficient for most needs. Responses can be tailored to return only what is required.	Excellent; a single request is usually sufficient for most needs. Responses only include exactly what was requested.
<u>Documentation, API explorability and schema</u>	Poor; no schema, not explorable.	Acceptable; generic schema only; links and error messages are self-documenting.	Excellent; precise schema; excellent tooling for exploration and documentation.
<u>Operational simplicity</u>	Acceptable; works out of the box with CDNs and reverse proxies; few to no client-side libraries required.	Excellent; works out of the box with CDNs and reverse proxies, no client-side libraries needed, but many are available and useful.	Poor; extra infrastructure is often necessary client side libraries are a practical necessity, specific patterns required to benefit from CDNs and browser caches.
<u>Writing data</u>	Acceptable; HTTP semantics give some guidance but how specifics left to each implementation, one write per request.	Excellent; how writes are handled is clearly defined by the spec, one write per request, but multiple writes is being added to the specification.	Poor; how writes are handled is left to each implementation and there are competing best practices, it's possible to execute multiple writes in a single request.

Based on this analysis, for Drupal core's needs, we rank JSON:API above GraphQL and GraphQL above REST. As such, I want to change my recommendation for Drupal 8 core. Instead of adding both JSON:API and GraphQL to Drupal 8 core, I believe only JSON:API should be added. That said, Drupal's GraphQL implementation is fantastic, especially when you have the developer capacity to build a bespoke API for your project.

Source

1.3.2 Problèmes de routing

Dans une architecture découplée, la gestion des routes entre le CMS (Drupal) et le front-end devient complexe, car il n'existe plus de relation directe entre les URLs et le rendu des pages. Cela crée plusieurs problèmes liés à la résolution et à la gestion des routes.

Résolution des routes

Problème : Le front-end ne sait pas automatiquement quelle entité Drupal correspond à une URL donnée.

Solution : Utilisation du module **Decoupled Router** qui permet de résoudre une URL en entité Drupal via une API, afin que le front-end puisse récupérer directement le bon contenu.

Gestion des alias d'URL

Problème : Lorsqu'une URL change dans Drupal, les liens côté front-end peuvent devenir invalides.

Solution : Utilisation des **URL aliases** et des **redirects** pour maintenir la compatibilité entre anciennes et nouvelles URLs.

Routing multi-frontends

Problème : Plusieurs applications (web, mobile, etc.) peuvent nécessiter des structures de routes différentes.

Solution : Utilisation de modules comme **Consumers** pour adapter les réponses API selon le type de client.

Résumé

1.3.3 Image Styles en Architecture Drupal Découplée

Décalage entre front-end et back-end

Problème : Les styles d'images dépendent du design (front-end), mais le front-end ne peut pas toujours les générer à cause de ses limites techniques (performance, bande passante, outils).

Solution : Le traitement des images est effectué côté Drupal, qui génère des versions optimisées et les fournit via l'API.

Trop grand nombre de variantes d'images exposées

Problème : Exposer toutes les variantes d'images pour tous les consommateurs génère des URLs inutiles, augmente les risques de sécurité et complique la gestion.

Solution : Utiliser le module **Consumers** pour déclarer les besoins de chaque application et limiter la génération des styles uniquement à ceux nécessaires.

Sécurité des styles d'images

Problème : En découplé, le back-end ne connaît pas les designs du front-end, donc il ne peut pas valider automatiquement les styles d'images utilisés.

Solution : Le module **Consumer Image Styles** permet d'associer des styles d'images à chaque consommateur et de sécuriser les URLs générées via des tokens.

Gestion des consommateurs inconnus

Problème : L'API peut être utilisée par plusieurs applications inconnues à l'avance, ce qui rend difficile la gestion des besoins spécifiques.

Solution : Le module **Consumers** permet d'enregistrer chaque application comme un consommateur avec sa propre configuration.

Limites du front-end

Problème : Le front-end peut être limité en puissance, en stockage et en bande passante pour traiter les images.

Solution : Drupal centralise le traitement des images et fournit directement des versions optimisées via l'API.

Le consommateur reçoit uniquement les variantes qu'il a déclarées, sans exposer les configurations d'autres consommateurs ou générer des images inutiles.

1.3.4 Schémas d'API en Architecture Drupal Découplée

Inflexibilité de la sérialisation

Problème : Le processus de sérialisation transforme les données Drupal en JSON, mais perd les informations sur leur structure et leur type, ce qui empêche de générer un schéma fiable automatiquement.

Solution : Utiliser le module **Schemata** pour générer automatiquement des schémas JSON à partir des données Drupal. Ces schémas peuvent ensuite être exposés et utilisés via **OpenAPI** pour améliorer la compréhension et la documentation de l'API.

Décalage entre schémas et exécution

Problème : Les schémas sont statiques alors que la sérialisation est dynamique, donc les données réelles peuvent différer du schéma décrit.

Solution : Réduire la flexibilité des transformations ou renforcer la validation côté serveur et côté client en s'appuyant sur les schémas générés par **Schemata** et exposés via **OpenAPI**.

Perte d'information dans les normaliseurs

Problème : Les normaliseurs transforment les données sans documenter clairement les changements, rendant la structure finale difficile à prévoir pour le front-end.

Solution : Améliorer les normaliseurs pour exposer les informations de type, afin de permettre à **Schemata** de générer des schémas plus précis et exploitables dans la documentation **OpenAPI**.

Manque de documentation fiable

Problème : Sans schémas précis, le front-end doit deviner la structure des données ou tester l'API pour comprendre les réponses.

Solution : Générer automatiquement la documentation avec **OpenAPI** à partir des schémas fournis par **Schemata**, garantissant une documentation toujours synchronisée avec l'API.

Incohérence des données

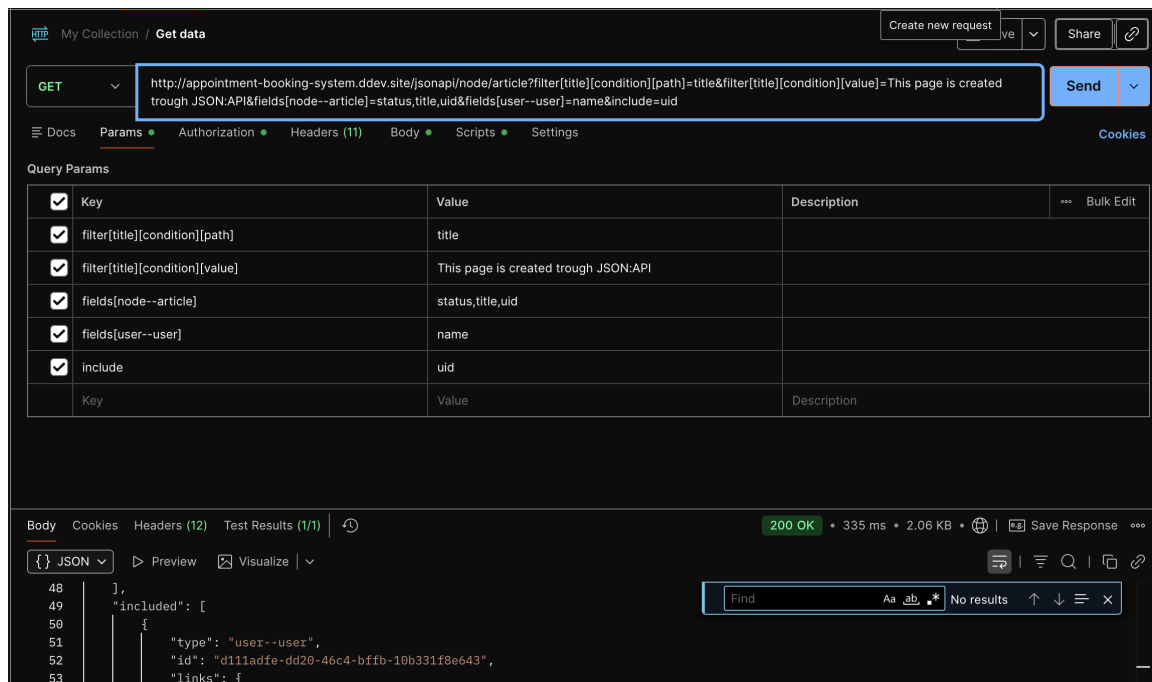
Problème : Sans validation stricte, les réponses API peuvent ne pas respecter le schéma attendu, ce qui crée des erreurs côté client.

Solution : Mettre en place des validations et des tests automatiques basés sur les schémas générés par **Schemata**, et exposés via **OpenAPI**, afin de garantir la cohérence des données.

1.4. Tâches à faire :

1.4.1 Exploration du module JSON :API avec Postman :

Dans cette partie, nous allons installer le module **JSON :API** et explorer son utilisation à travers **Postman** afin de tester les différentes fonctionnalités comme les champs, les relations (includes), le tri et les filtres.



- **fields[type]** : permet de limiter les données retournées en sélectionnant uniquement certains champs, afin de réduire la taille de la réponse.
- **filter[path][value]** : permet de filtrer les ressources selon des critères précis, y compris les champs liés à d'autres entités (relations).
- **[include]** : charge les données liées (utilisateur, médias, taxonomies, etc.) dans une seule requête pour éviter plusieurs appels API.
- **page[limit] et page[offset]** : permettent de gérer la pagination en définissant le nombre de résultats et le point de départ.
- **sort** : permet de trier les résultats selon un ou plusieurs champs (ordre croissant ou décroissant).
- **Endpoints de relations** : permettent d'accéder aux données des entités liées (par exemple l'auteur d'un article).
- **POST** : crée un contenu (node). Nécessite le header **Content-Type: application/vnd.api+json** et toutes les données obligatoires, sinon erreur **422 Unprocessable Content**.
- **PATCH** : met à jour un contenu existant. On envoie uniquement les champs à modifier, mais il faut utiliser le bon endpoint (langue correcte sinon erreur **405 Method Not Allowed**).
- **DELETE** : supprime un contenu via son UUID, sans body dans la requête.

1.4.2 Ajout et exposition d'un champ personnalisé via l'API :

Dans cette partie, nous allons créer un nouveau champ en utilisant `hook_entity_base_field_info`, puis l'exposer via l'API afin de le rendre accessible et exploitable dans les réponses JSON :API.

```

function blog_entity_base_field_info($entity_type) {
  // START: This function receives the entity type object
  // $entity_type = object representing the 'blog' entity

  $fields = [];

  // Only add fields for the blog entity (could add fields for other entities too)
  if ($entity_type->id() === 'blog') {

    // Create a field definition
    // Think of this as: "I want a field named 'summary' that is text_long type"
    $fields['summary'] = \Drupal\Core\Field\BaseFieldDefinition::create('text_long')
      ->setLabel(t('Summary')) // Human-readable label
      ->setDescription(t('A short summary for the blog.')) // What it's for
      ->setDisplayOptions('view', [ // How to display on page
        'type' => 'text_default',
        'label' => 'above',
        'weight' => 5,
      ])
      ->setDisplayOptions('form', [ // How to show in forms
        'type' => 'text_textarea',
        'weight' => 5,
      ])
      ->setDisplayConfigurable('form', TRUE) // Can admins configure this?
      ->setDisplayConfigurable('view', TRUE); // Can admins configure this?
  }

  // END: Return the array of field definitions
  // For new installs: Drupal uses this to know the field exists
  // For existing installs: This is just metadata, the field might not be in the DB yet
  return $fields;
}

```



```

function blog_update_11002() {

    // STEP 1: Get services we need to access entity and database definitions
    $entity_type_manager = \Drupal::entityTypeManager();
    $definition_update_manager = \Drupal::entityDefinitionUpdateManager();

    // STEP 2: Get the blog entity type definition
    // This gives us the entity object to pass to the hook
    $entity_type = $entity_type_manager->getDefinition('blog');

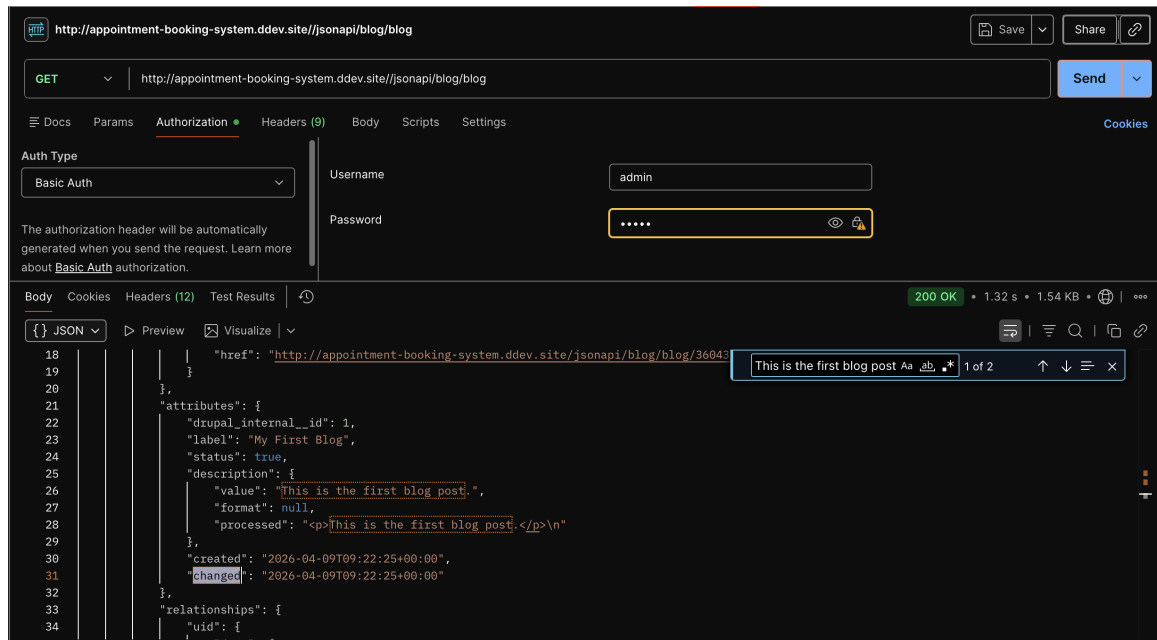
    // STEP 3: CALL THE HOOK FUNCTION ← THIS IS THE KEY RELATIONSHIP
    // This line retrieves the field definitions we defined in the hook above
    // It returns: ['summary' => BaseFieldDefinition object]
    $fields = blog_entity_base_field_info($entity_type);

    // STEP 4: Safety check - does the field exist in the hook?
    if (!isset($fields['summary'])) {
        throw new \Exception('Summary field not found in blog_entity_base_field_info()');
    }

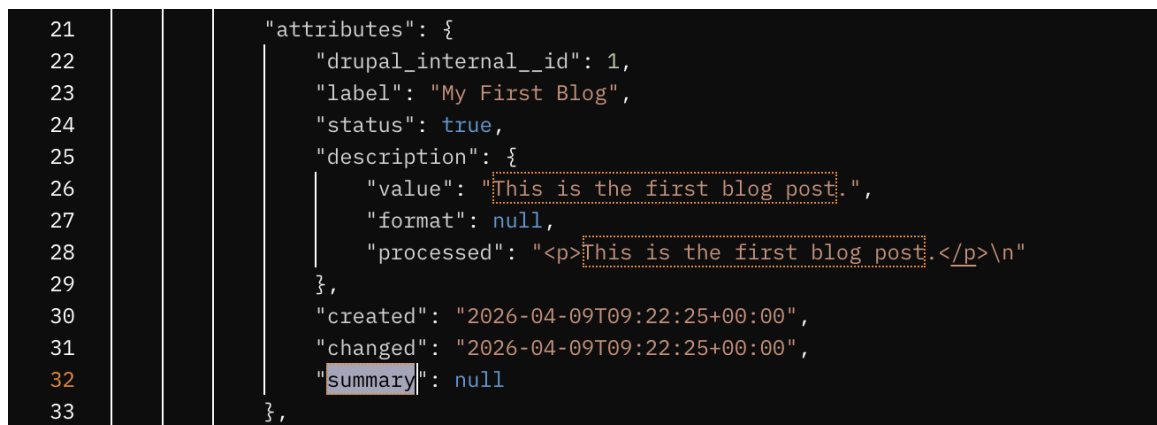
    // STEP 5: Get the specific field definition
    // $fields['summary'] is the BaseFieldDefinition object we created in the hook
    $summary_field_definition = $fields['summary'];

    // STEP 6: INSTALL it to the database
    // This creates the actual database tables/columns where data will be stored
    // It uses the definition we retrieved from the hook
    $definition_update_manager->installFieldStorageDefinition(
        'summary',          // Same name we used in the hook
        'blog',             // Entity type (same as hook)
        'blog',             // Module name providing this entity
        $summary_field_definition // The definition from the hook ← CRITICAL
    );
}

```



Après l'ajout du hook :



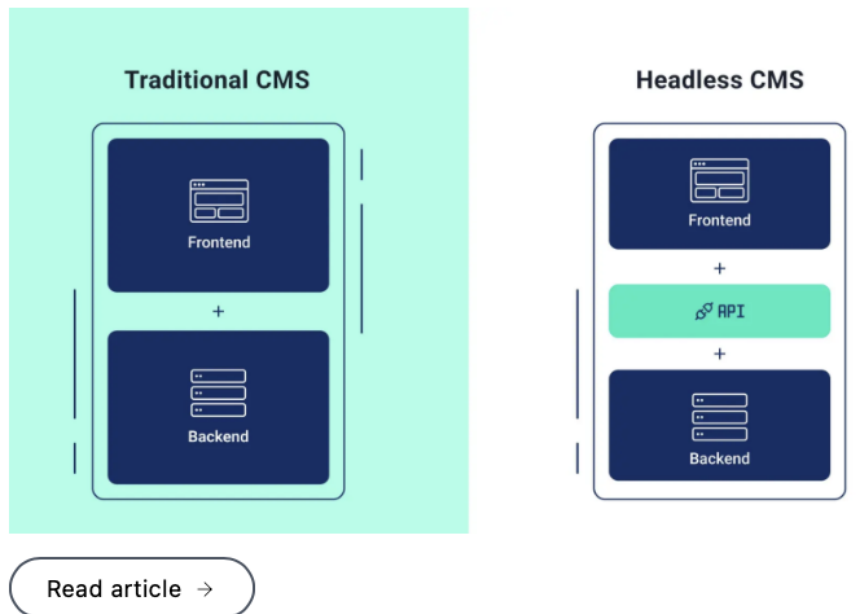
1.4.3 Mise en pratique avec Next.js et Drupal (Next-Drupal) :

Dans cet exercice, nous allons utiliser une nouvelle stack basée sur Next.js et Drupal afin de tester notre capacité d'adaptation. L'objectif est de suivre la documentation de Next-Drupal pour mettre en place un projet fonctionnel avec une page d'accueil et une page listant des articles récupérés depuis Drupal.

Latest Articles.

My second Article

Posted by admin - April 9, 2026



[Read article →](#)

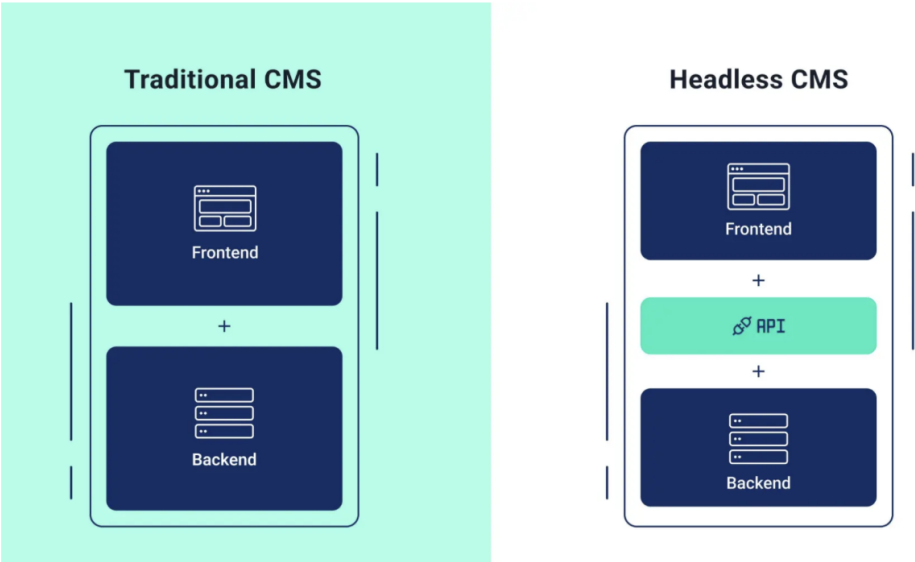
My first article

Posted by admin - April 9, 2026

The screenshot shows the 'La Factory 10.2.3' configuration interface. On the left is a sidebar with a list of steps: 'Verify requirements', 'Set up database', 'Install site', 'Configure site' (highlighted with a grey arrow), 'Configure multilingual', 'Configure additional modules', 'Install additional modules', and 'Import nodes'. The main area is titled 'Configure site' and contains a 'SITE INFORMATION' section. It has two input fields: 'Site name' with the value 'Vactory' and 'Site email address' with the value 'admin@void.fr'. Below the email field is a note: 'Automated emails, such as registration information, will be sent from this address. Use an address ending in your site's domain to help prevent these emails from being flagged as spam.'

My second Article

Posted by admin - April 9, 2026



describing the article

1.5. Questions :

Contexte : La Vactory

— [Comment récupérer les blocs d'une page en utilisant JSON :API?](#)

node--vactory_page /api/node/vactory_page Overwritten Edit

Les types content block :

Block description	Block type	Updated	Operations
Default footer	Template	04/10/2026 - 16:03	Edit
Default header	Template	04/10/2026 - 15:32	Edit

L'api suivante expose all the blocks :

HTTP <http://localhost:8000/api/block/block> [Save] [Share]

GET <http://localhost:8000/api/block/block> [Send]

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Key	Value	Description	Bulk Edit
filter[block][condition][path]	internal_blocks.id		
filter[block][condition][operator]	IN		
filter[block][condition][value]	vactory_decoupled_default_footer		
Key	Value	Description	

Body Cookies Headers (19) Test Results 200 OK • 150 ms • 11.91 KB

```

444     "container_spacing": "small_space"
445   },
446 },
447   "drupal_internal_id": "vactory_default_header",
448   "theme": "vactory",
449   "region": "header",
450   "weight": -2,
451   "provider": null,
452   "plugin": "block_content:3283d092-b069-4c7b-8ef9-d44fc76d6ff6",
453   "settings": {
454     "id": "block_content:3283d092-b069-4c7b-8ef9-d44fc76d6ff6",
455     "label": "Default header",
456     "label_display": "visible",
457     "provider": "block_content",
458     "status": true,

```

Pour une seule page :

HTTP http://localhost:8000/api/node/vactory_page/2e5d9998-32c2-42d2-ac89-da599697ea9c [Save] [Share]

GET [http://localhost:8000/api/node/vactory_page/2e5d9998-32c2-42d2-ac89-da599697ea9c?include=field_vactory_paragraphs&fields\[node--vactory_page\]=internal_blocks](http://localhost:8000/api/node/vactory_page/2e5d9998-32c2-42d2-ac89-da599697ea9c?include=field_vactory_paragraphs&fields[node--vactory_page]=internal_blocks) [Send]

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
☒ include	field_vactory_paragraphs		
☒ fields[node--vactory_page]	internal_blocks		
filter[block][condition][path]	internal_blocks.id		

Body Cookies Headers (19) Test Results 200 OK • 486 ms • 5.3 KB

```

20   "attributes": {
21     "internal_blocks": [
22       {
23         "id": "vactory_decoupled_default_footer",
24         "label": "Vactory Decoupled Default Footer",
25         "region": "footer",
26         "plugin": "block_content:3e9050c3-8e9d-419e-b0f9-31174ff0f520",
27         "weight": -3,
28         "classification": "default",
29         "content": {
30           "widget_id": "vactory_footer:footer-variant1",
31           "widget_data": "{\n  \"auto_populate\": false,\n  \"template\": \"vactory_footer:footer-variant1\"\n}"
32         },
33         "classes": null,

```

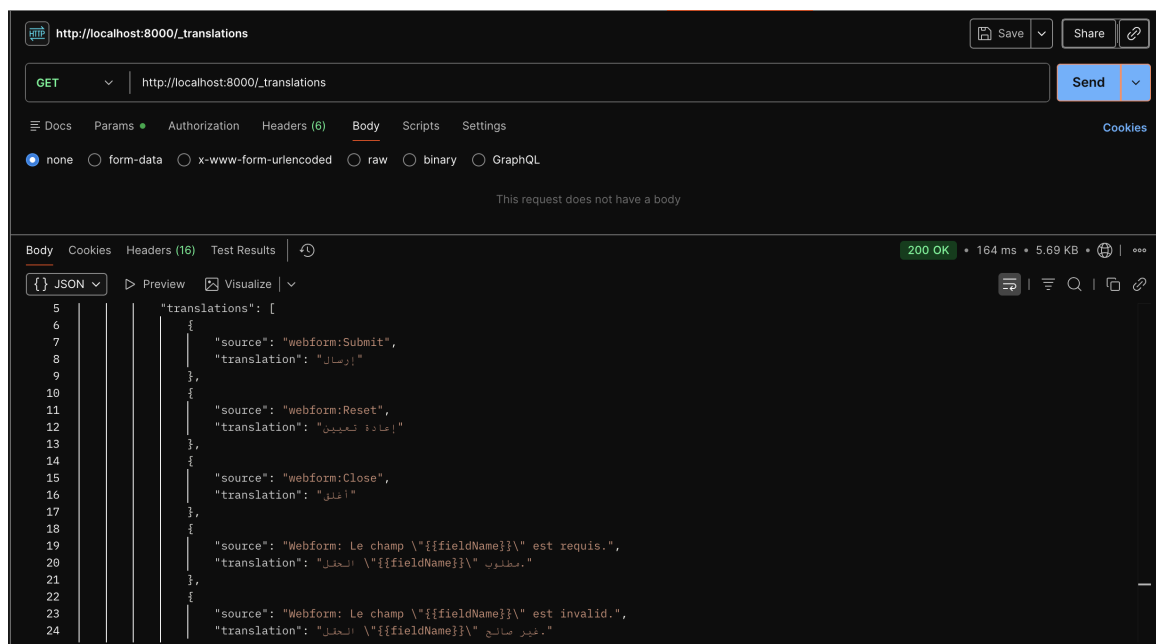
— Quel est l'endpoint utilisé pour récupérer les traductions et les menus ?

Dans le module : *vactory_decoupled_router*

— Les translations

Dans le fichier : *vactory_decoupled.routing.yml*

```
vactory_decoupled.translations:
  path: '/_translations'
  defaults:
    _controller: '\Drupal\vactory_decoupled\Controller\TranslationsController::index'
    _disable_route_normalizer: 'TRUE'
  methods: [GET]
  requirements:
    _permission: 'access content'
    _format: 'json'
```



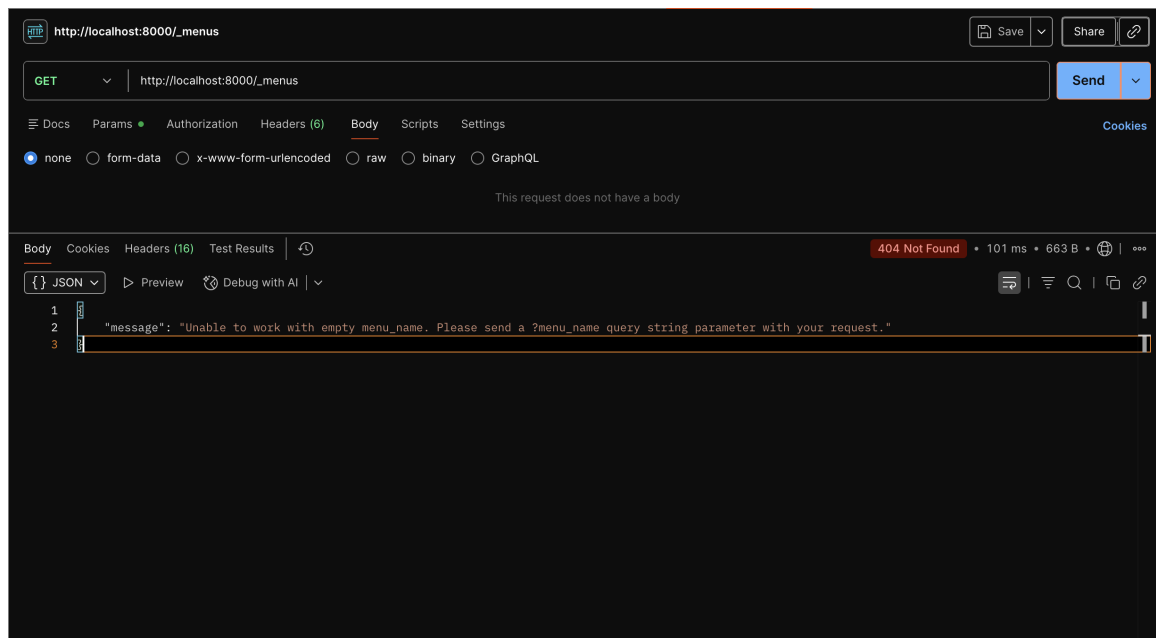
The screenshot shows a web browser interface with a GET request to `http://localhost:8000/_translations`. The response is a JSON array of translations. The interface includes tabs for Docs, Params, Authorization, Headers (6), Body, Scripts, and Settings. The Body tab is selected, showing the JSON response. The response status is 200 OK, with a response time of 164 ms and a size of 5.69 KB. The JSON response is as follows:

```
{
  "translations": [
    {
      "source": "webform:Submit",
      "translation": "إرسال"
    },
    {
      "source": "webform:Reset",
      "translation": "إعادة تعيين"
    },
    {
      "source": "webform:Close",
      "translation": "إغلاق"
    },
    {
      "source": "Webform: Le champ \"{{fieldName}}\" est requis.",
      "translation": "مطلوب \"{{fieldName}}\" الحقل."
    },
    {
      "source": "Webform: Le champ \"{{fieldName}}\" est invalid.",
      "translation": "غير صالح \"{{fieldName}}\" الحقل."
    }
  ]
}
```

— Les menus

Dans le fichier : *vactory_decoupled.routing.yml*

```
vactory_decoupled.menus:
  path: '/_menus'
  defaults:
    _controller: '\Drupal\vactory_decoupled\Controller\MenusController::index'
  methods: [GET]
  requirements:
    _permission: 'access content'
    _format: 'json'
```



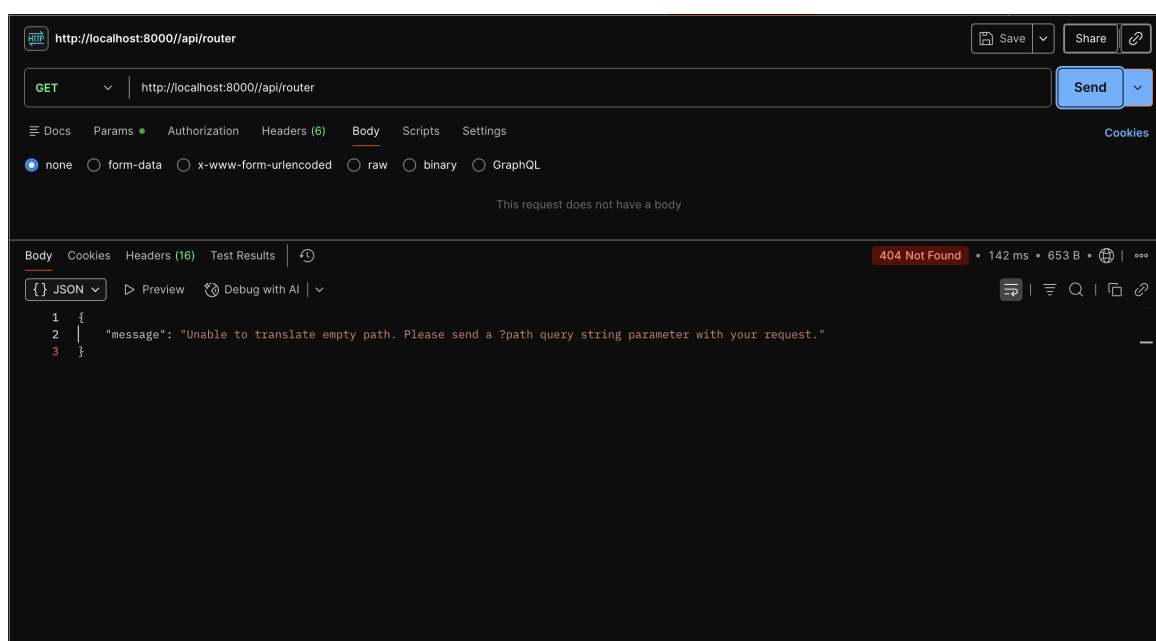
— Quel est l'endpoint utilisé pour traduire une URL ou résoudre un alias ?

Dans le module : *vactory_decoupled_router*

```

1  vactory_decoupled_router.path_translation:
2  ✦path: '/api/router'
3  defaults:
4    _title: 'Translates an incoming path into a formatted response.'
5    _controller: '\Drupal\vactory_decoupled_router\Controller\PathTranslator::translate'
6    _disable_route_normalizer: 'TRUE'
7  methods: [GET]
8  # options:
9  #   _auth: ['basic_auth', 'cookie', 'oauth2']
10 requirements:
11   _permission: 'access content'
12   _format: json
13

```



+ Add route

Machine name	Route	Path	Alias	Status	Operations
account	Account	/node/2	/account	✓	Edit ▼
account_verify	Account Verify	/node/8	/account/verify	✓	Edit ▼
error_page	Error Page	/node/1	/error-page	✓	Edit ▼
account_forget_password	Forget Password	/node/7	/account/forget-password	✓	Edit ▼
account_login	Login	/node/3	/account/login	✓	Edit ▼
account_lost_password	Lost Password	/node/5	/account/lost-password	✓	Edit ▼

— Quel module est responsable de la création des pages suivantes :

- Page d'erreur (/error-page)
- Compte utilisateur (/account)
- Connexion (/account/login)
- Inscription (/account/register)
- Mot de passe oublié (/account/lost-password)
- Réinitialisation du mot de passe (/account/reset-password)
- Demande de réinitialisation (/account/forget-password)
- Vérification de compte (/account/verify)

C'est le module : *vactory_decoupled_router_default*


```

*/
function _get_vactory_decoupled_router_default_pages() {
  return [
    'error_page' => [
      'path' => '/error-page',
      'title' => 'Error Page',
    ],
    'account' => [
      'path' => '/account',
      'title' => 'Account',
    ],
    'account_login' => [
      'path' => '/account/login',
      'title' => 'Login',
    ],
    'account_register' => [
      'path' => '/account/register',
      'title' => 'Register',
    ],
    'account_lost_password' => [
      'path' => '/account/lost-password',
      'title' => 'Lost Password',
    ],
    'account_reset_password' => [
      'path' => '/account/reset-password',
      'title' => 'Reset Password',
    ],
    'account_forget_password' => [
      'path' => '/account/forget-password',
      'title' => 'Forget Password',
    ],
    'account_verify' => [
      'path' => '/account/verify',
      'title' => 'Account Verify',
    ],
  ],
};

```

Ce hook est appelé lors de l'installation par le hook suivant :

```

/**
 * Implements hook_install().
 */
function vactory_decoupled_router_default_pages_install() {
  foreach (_get_vactory_decoupled_router_default_pages() as $machine_name => $page) {
    $nodes = \Drupal::entityTypeManager()
      ->getStorage('node')
      ->loadByProperties(['machine_name' => $machine_name]);

    if (empty($nodes)) {
      _vactory_decoupled_router_create_default_node($machine_name, $page['title'], $page['path']);
    }
  }
}

// Run a complete cache flush.
drupal_flush_all_caches();
}

```

Filter

Vactory Decoupled Router Default Pages

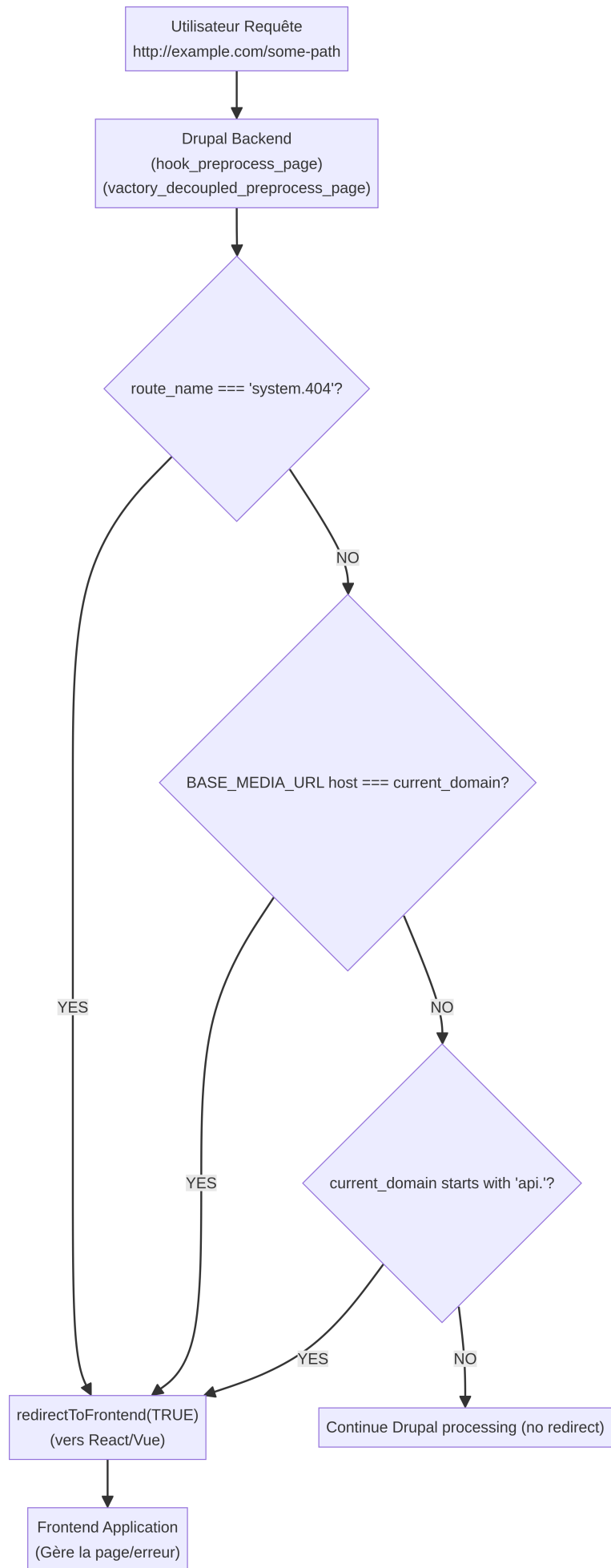
^ Vactory

☒ Vactory Decoupled Router Default Pages

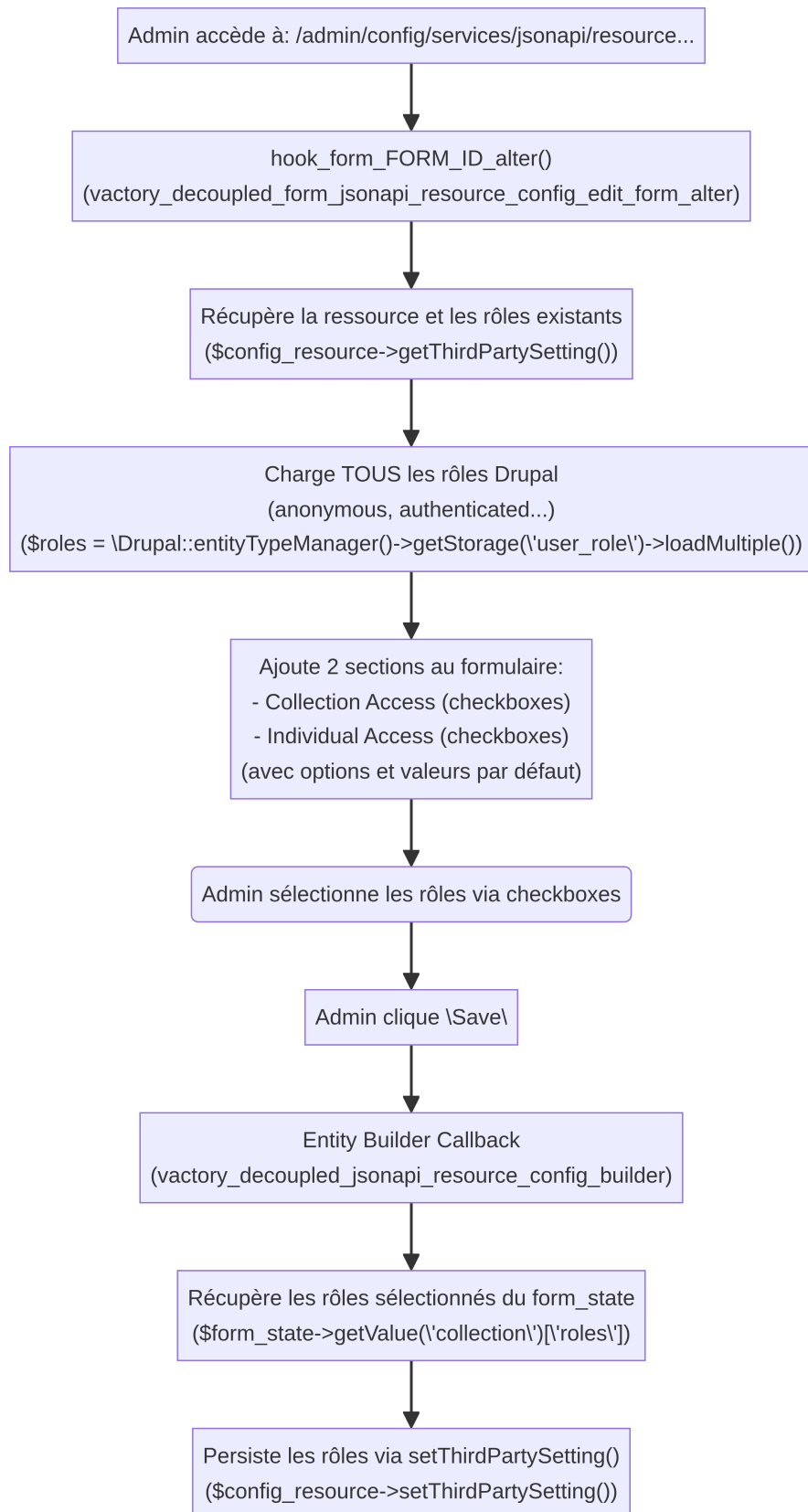
☐ Create pages and routes.

Install

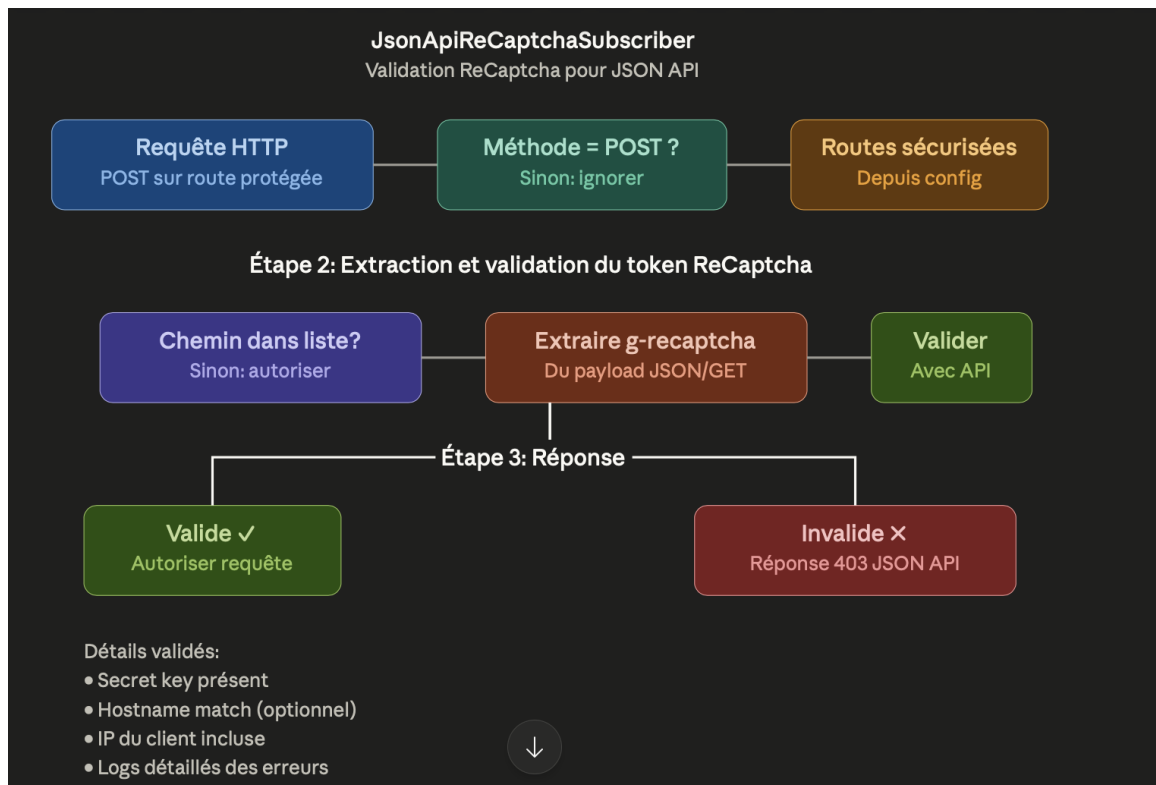
— Que fait `vactory_decoupled_preprocess_page`?



— Que fait `vactory_decoupled_form_jsonapi_resource_config_edit_form_alter` ?

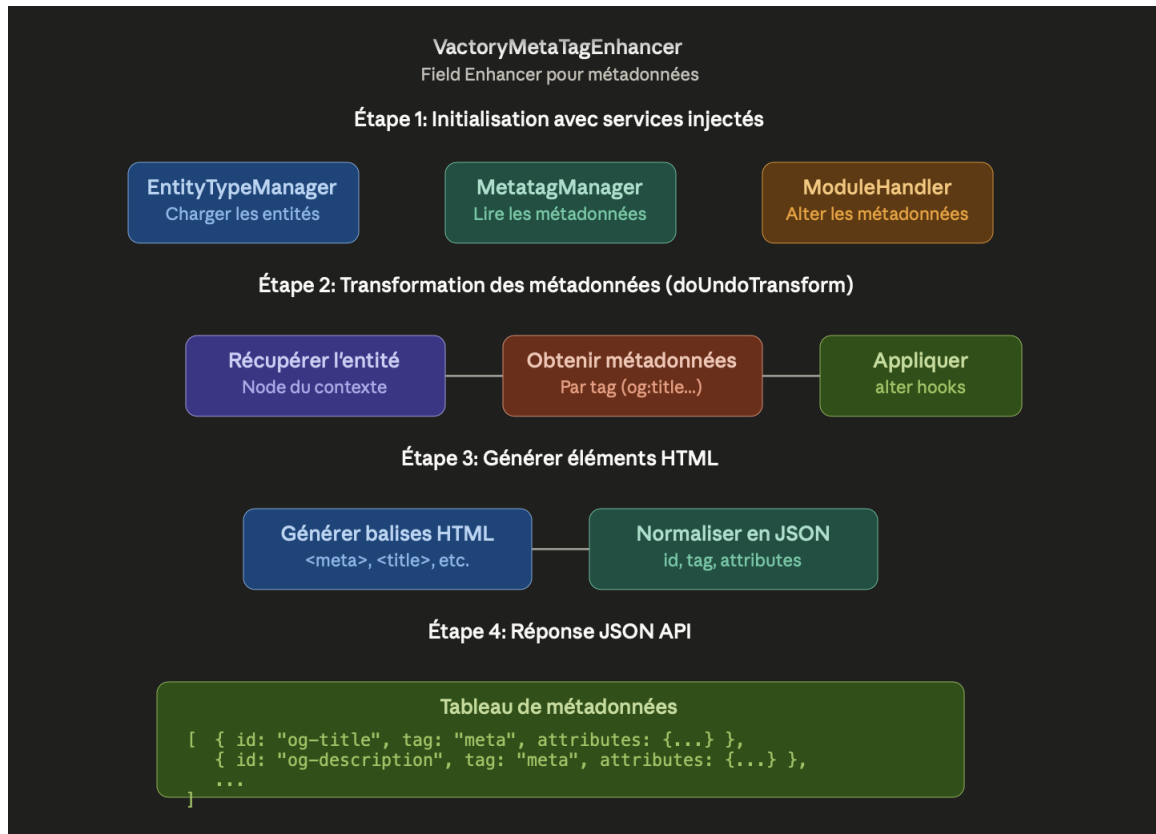


— Quel est le rôle de `JsonApiReCaptchaSubscriber.php` ?



— Quel est le rôle de `VactoryMetaTagEnhancer.php` ?

Ce fichier PHP crée un Field Enhancer pour JSON API qui expose les métadonnées Drupal (Open Graph, Dublin Core, etc.) sous forme d'un tableau JSON propre et exploitable par le front-end.



— Quel est le rôle de `InternalNodeEntityBlocksFieldItemList.php` ?

Ce fichier crée un champ calculé qui récupère dynamiquement tous les blocs Drupal associés à une Node et

les expose via JSON API. C'est un champ "fantôme" qui n'existe pas en base de données mais est généré à la demande.

